

Counting Polyominoes, Revisited

Gill Barequet

`barequet@cs.technion.ac.il`

Technion – Israel Institute of Technology

Gil Ben-Shachar

Technion – Israel Institute of Technology

Research Article

Keywords: Polyominoes , lattice animals , counting algorithms

Posted Date: May 6th, 2024

DOI: <https://doi.org/10.21203/rs.3.rs-4304962/v1>

License:   This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

Counting Polyominoes, Revisited

Gill Barequet · Gil Ben-Shachar

Received: date / Accepted: date

Abstract A *polyomino* is an edge-connected set of squares on the square lattice. In this paper, we improve Jensen’s algorithm for counting polyominoes by considering bounding boxes on the square lattice rotated by 45° instead of on the regular unrotated lattice. This allows us to extend significantly the count of polyominoes from 56 to 70 terms.

Keywords Polyominoes · lattice animals · counting algorithms

1 Introduction

Polyominoes of size n are edge-connected sets of n cells on the square lattice. Methods for counting them have been studied for more than 60 years in statistical physics in the context of percolation processes [5], and this problem has since then been an active research topic in enumerative combinatorics. Counting polyominoes was stated as Problem 37 in The Open Problems Project (see <https://topp.openproblem.net/p37>). In the mathematics literature, the notation $A(n)$ (Sequence A001168 in the On-line Encyclopedia of Integer Sequences [14]) is commonly used for the number of *fixed* polyominoes, that is, polyominoes that are distinct up to translations.

More than half a century ago, Klarner [10] proved that the limit $\lambda = \lim_{n \rightarrow \infty} \sqrt[n]{A(n)}$ exists, henceforth coining λ , the growth constant of $(A(n))$,

Work on this paper has been supported in part by BSF Grant 2017684. A preliminary version of this paper [1] appeared in ALENEX’24.

G. Barequet*

Dept. of Computer Science, The Technion—Israel Inst. of Technology, Haifa 3200003, Israel.
E-mail: barequet@cs.technion.ac.il

*Corresponding author

G. Ben-Shachar

Dept. of Computer Science, The Technion—Israel Inst. of Technology, Haifa 3200003, Israel.
E-mail: gilbe@cs.technion.ac.il

as “Klarner’s constant.” The convergence of $A(n+1)/A(n)$ to λ , as n tends to infinity, was proved only three decades later by Madras [12]. It is widely believed that $\lambda \approx 4.06$, and the currently best estimate, $\lambda = 4.0625696 \pm 0.0000005$, is due to Jensen [9]. The best-known lower and upper bounds on λ are 4.0025 [3] and 4.5252 [4], respectively.

Early algorithms counted polyominoes by generating them systematically, however, with repetitions, hence most of the running time of these algorithms was spent on identifying and eliminating duplicates. The first efficient algorithm for counting polyominoes was presented in 1981 by Redelmeier [13]. This algorithm, based on a method for enumerating connected subgraphs in a graph, counts polyominoes by generating them one by one *without repetitions*. It was then used to count polyominoes up to size $n = 24$. The second-generation algorithm, suggested by Jensen [8,9] (see also Ref. [7, Appendix]), provided a listing of $A(n)$ up to $n = 56$. This algorithm was a specialization of an algorithm by Conway [6], which was used for enumerating percolation series. The genuine contribution of this algorithm was in counting polyominoes *without* actually producing them. In this paper, we present a significant improvement to Jensen’s algorithm, and extend considerably the known count of polyominoes up to size 70.

The paper is organized as follows. In Section 2, we briefly describe the main ingredients of Jensen’s algorithm. Then, in Section 3, we describe our improvement to the algorithm, and detail in Section 4 further modifications to it. Section 5 compares between the two versions of the algorithm. In Section 6, we provide the new counts of polyominoes. We end in Section 7 with some concluding remarks.

2 The Algorithm of Jensen

We provide here the main ingredients of Jensen’s algorithm. The interested reader is referred to Refs. [8,9] for more details.

The polyomino-counting algorithm of Jensen is based on a transfer-matrix method which is efficient mainly because it does not produce all polyominoes. Instead, it maintains all possible *right boundaries* of polyominoes partly-built within axis-aligned boxes. (Here, “partially built” means that during the course of the algorithm, polyominoes are possibly made of yet disconnected components). The algorithm maintains, for every possible boundary, information about all options for the connectivity (that is, the existence of a path of neighboring cells) between cells of the boundary through cells found to the left of it. The composition of a boundary and a connectivity is called a “signature.” Connectivity of the boundary cells is encoded by a 5-letter alphabet: ‘0’ encodes an empty cell; ‘1’ encodes a singleton boundary cell; and if at least two boundary cells belong to the same component, ‘2’ encodes the highest, ‘4’ encodes the lowest, and ‘3’ encodes all the other boundary cells (if exist) that belong to this component. Figure 1 shows a polyomino whose signature (from

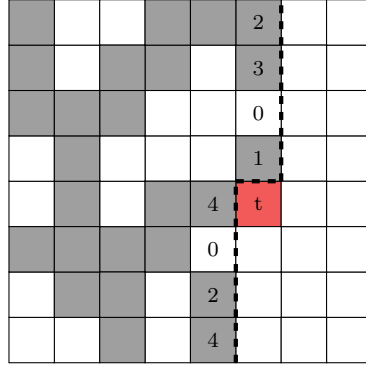


Fig. 1 A partly-built 25-cell polyomino with signature “23014024”. The current boundary is marked with a dashed line and the next cell to be added, t , is colored in red.

top to bottom)¹ is “23014024”.

Polyominoes are imaginarily built, column by column, from left to right, and in every column, squares are considered, one by one, from top to bottom. The algorithm maintains a database whose keys are the signatures, and for each possible signature σ , the corresponding entry in the database contains the *counts* of partial polyominoes, of all possible sizes, that have boundaries with signature σ . Note that the database stores only these statistics but does not store all possible polyominoes. Therefore, the main factor that dominates the memory consumption and the running time of the algorithm is the *number of signatures* and not the number of polyominoes. Both numbers are exponential with n , but as we show below, the base of the former quantity is much smaller than that of the latter. All other factors that influence memory and time are much less significant. This gives a huge advantage to Jensen’s algorithm over that of Redelmeier [13] since the latter produces all polyominoes.

The basic step of the algorithm considers the next cell t (refer again to Figure 1), located just below the “kink” of the boundary. In every such step, there are two options: Either adding or not adding cell t to the polyomino. After the kink moves below cell t , either option changes the signature from, say, σ to another signature σ' . The algorithm then updates the contents of the database entry with key σ' , using the contents of the entry with key σ .

During the course of the algorithm, it sums up the counts found in all entries that correspond to valid (connected) polyominoes up to some prescribed size and for all possible heights (y spans). Note that whether or not partial polyominoes are connected is manifested by their signatures σ . This is done when the boundary “loses contact” with the polyominoes to the left of it, that is, when the boundary σ' becomes empty.

Remarks.

1. The algorithm needs only be run for all heights $1 \leq h \leq \lceil n/2 \rceil$ since all polyominoes whose height exceeds $\lceil n/2 \rceil$ are also detected with their 90°-

¹ In Jensen’s notation, signatures are specified bottom to top.

rotated versions. Hence, the algorithm counts only polyominoes whose width is at least their height, and doubles the counts of polyominoes whose width is strictly bigger than their height.

2. However, in order to avoid multiple counting of polyominoes whose height is strictly less than $\lceil n/2 \rceil$ (since they fit into more than one box), the keys (signatures) are extended with two bits, being set, respectively, for polyominoes touching the floor and the ceiling of the bounding box. (Note that these two indicators are not included in the original signatures.) Thus, polyominoes are counted only for entries whose two additional bits are set.

As mentioned above, analysis of the algorithm [2] shows that the major factor in its running time is the number of signatures. It turns out that the number of possible signatures of length h is roughly proportional to 3^h . (This is proven by a bijection between signatures and Motzkin paths.) The amount of storage in each entry of the database, and the time needed for accessing and processing each entry in the database, are proportional to low-degree polynomials in n . Hence, the running time of the algorithm is roughly (up to a small polynomial factor) $3^{n/2} \approx 1.73^n$, a quantity that is significantly smaller than 4.06^n , which is roughly the number of polyominoes of size n . Pruning of signatures (discarding signatures of partial polyominoes that cannot grow and form valid polyominoes) is a very efficient enhancement of the algorithm. Without pruning, the higher the bounding box is, the more time-consuming its processing is. (This is implied immediately by the fact that the number of signatures depends exponentially on the height of the box.) It turns out that when signatures are pruned, the height of the box that is the most computationally heavy becomes less than $n/2$, and for higher boxes, pruning is more effective and the running time for processing them drops significantly. Hence, at least for small values of n , when pruning is used, the running time behaves even better than $3^{n/2}$. As we describe below, we introduce a crucial improvement to Jensen's algorithm, and thereby compute many more terms of the sequence $(A(n))$, extending the count by 14 new terms.

3 Our Improvement

Jensen's algorithm essentially computes the number of n -cell polyominoes bounded by axis-aligned boxes of various heights (originally called "widths"). In a nutshell, our improvement is achieved by rotating the bounding boxes by 45° and scanning it along diagonals. (Alternatively, we rotate the *lattice* by 45° and scan it by columns.)

For describing our modification, we have to define the *diagonal height* of a polyomino. First, we need to assign sequential ordinal numbers to the diagonals of the lattice. Figure 2(a) shows the numbering of descending and ascending diagonals. Then, we need to assign to every cell coordinates that are based on this numbering. Formally, we associate with every cell (x, y) the two diagonals it resides on using the functions $D_1(x, y) = x + y$ and $D_2(x, y) = x - y$, that is, $(D_1(x, y), D_2(x, y))$ are the cell's coordinates in the rotated lattice. The

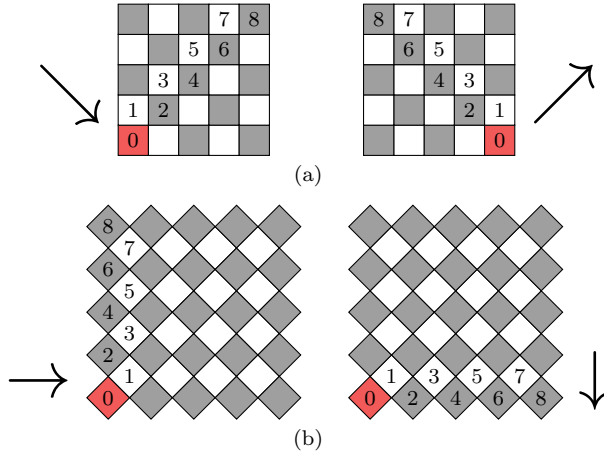


Fig. 2 The numbering of diagonals (the red cell is the origin (0,0)): (a) Left: Descending diagonals (D_1); Right: Ascending diagonals (D_2). (b) The same diagonals on the rotated lattice, becoming rows and columns, respectively.

two diagonal distances (horizontal and vertical) between two cells, (x_1, y_1) and (x_2, y_2) , are $|D_1(x_1, y_1) - D_1(x_2, y_2)|$ and $|D_2(x_1, y_1) - D_2(x_2, y_2)|$, respectively.

For ease of exposition, we will hereafter draw the lattice rotated by 45° as in Figure 2(b). In this orientation, each original diagonal becomes a row or a column, and so the terms “diagonal width” (or “height”) of a polyomino are defined as the maximum horizontal (or vertical) distance between two cells in the rotated lattice, plus one.

Let us now define *axis-aligned boxes* on the rotated lattice. A box of width w and height h contains all cells found in between and including two columns within distance w and two rows within distance h . In contrast with boxes on the regular (unrotated) lattice, the shape of such a box may take several forms, depending on the parities of the columns and rows of its boundary; See Figure 3 for a few examples.

Similarly to the original algorithm, our version counts the different polyominoes that span the entire box, *i.e.*, touch its four sides, for all possible sizes of axis-aligned boxes. An alternative interpretation of a polyomino spanning a box is saying that it has the same diagonal height and width (regular height and width on the rotated lattice) as the box it spans. Figure 4 shows sample polyominoes which span the different boxes shown in Figure 3.

As in the original algorithm, we simulate the process of building polyominoes cell by cell, while keeping the statistics of partially-built polyominoes for each possible boundary signature. Cells on the boundary are classified according to the components they belong to in the same manner as in the original algorithm. An example of a partial polyomino and the respective signature is shown in Figure 5. Like in the original algorithm, we say that the “kink” of the signature is in the transition of the boundary between a column to the

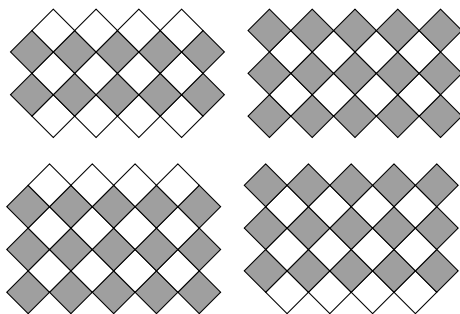


Fig. 3 A few types of boxes. The two top boxes are of height 5 and width 9, however, they are essentially different. The two bottom boxes are of height 6 and width 9; these boxes are different as well but one can be obtained from the other by mirroring about the x direction.

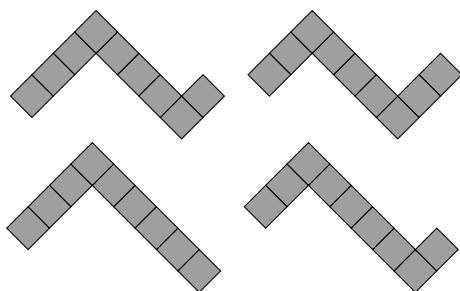


Fig. 4 Polyominoes which span the entire corresponding boxes in Figure 3.

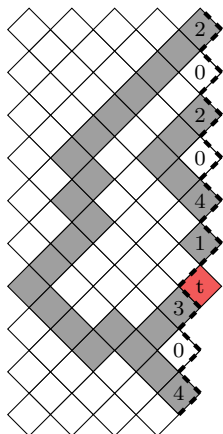


Fig. 5 A sample partial polyomino P , with signature “202041304”, built in the course of the algorithm. The signature of P is the sequence of numbers on its boundary. The current boundary is marked with a dashed line and the next cell to be considered, t , is colored in red.

Table 1 The modified transfer matrix (actions taken when processing the next cell t , the cell under the kink).

U \ L	0	1	2	3	4
0	00 10	01 24	02 23	03 33	04 34
1	add 10	— 24	— 23	— 33	— 34
2	$\overline{00}$ 20	$\overline{01}$ 23	$\widehat{02}$ 23	02 23	01 24
3	00 30	01 33	02 $\widehat{33}$	03 33	04 34
4	$\overline{00}$ 40	$\overline{01}$ 34	$\overline{02}$ 33	$\overline{03}$ 33	$\overline{04}$ 34

column immediately to the left of it. In Figure 5, the kink is above and to the left of the red cell t . When cell t is considered, the types of the cells to the left and above it, as well as whether the next cell is occupied or empty, imply the modified signature. Such operations are stated as “transition rules.” Obviously, the transition rules in the modified algorithm are different from those in the original algorithm. The modified rules are shown in Table 1. In this table, the left column and the top row specify, respectively, the types of the cell on top (U) and on the bottom-left side (L) of the kink cell t (colored in red in Figure 5). Each entry corresponds to a pair of cells (U and L) around the kink, and it contains two actions, the left (resp., right) to be performed when the considered kink cell becomes empty (resp., full). Each operation is made of two digits: The types of the two cells around the new kink. However, for certain configurations (U and L cells, and whether the considered cell becomes empty or full), the operation is not local but rather requires the scanning of the entire boundary and potentially updating the types of other cells along it. There are two types of such global actions, denoted by a bar or a hat above the 2-digit action. A bar denotes the case in which the first or last cell of a connected component disappears from the boundary, and thus, another cell needs to be marked as the first or last cell in this component. The hat sign denotes the case in which two different components are united into one component, thus, the signature needs to be adjusted accordingly. (See Ref. [9] for the full details.) The action “add” means that the statistics of the current signature should be added to the total count if the respective polyominoes span the entire height of the box. (Recall the two bits of the signature that indicate this.)

4 Algorithm Modifications

4.1 Pruning

A key idea for the efficiency of the original algorithm of Jensen is the pruning of signatures representing partial polyominoes that cannot be completed into polyominoes which will eventually be counted. Recall that in order for a final signature to represent valid (connected) polyominoes, it needs to (a) Represent boundaries of shapes that have only one components; (b) Touch the top and bottom of the box; and (c) Represent polyominoes that are wider than their height. The first requirement is for counting only valid (connected) polyomi-

noes, and the second and third requirements are needed for avoiding multiple counting of polyominoes. As we explain below, some intermediate signatures cannot evolve into final signatures that represent boundaries of valid polyominoes, and, hence they can be discarded in order to save running time later.

Let us consider a specific signature σ , and denote by $n_c = n_c(\sigma)$ the minimum number of additional cells that are needed for transforming any partially-built polyomino with signature σ into some connected polyominoes. Similarly, denote by n_h (resp., n_w) the minimum number of additional cells needed to make partial polyominoes with signature σ touch the top and bottom rows (resp., to be wider at least as their height). Obviously, depending on the signature, each of these three values can be 0. In the original algorithm, these three quantities are independent, thus, if the target size, up to which we want to count polyominoes, is n , we can discard any entry in the database that is associated with a signature that corresponds to partial polyominoes of size $n - (n_c + n_h + n_w) + 1$ (or more) since completing it into valid polyominoes would result in polyominoes of sizes greater than n . We emphasize that setting a target size n is not necessary for running the algorithm, however, it is necessary for pruning the signatures, an ingredient that speeds up the algorithm since the exponential number of signatures is the main factor in its running time.

Let us now add the important observation that in our version, the three quantities mentioned above are not independent, and cells that will contribute to turning partial polyominoes into valid ones (later in the course of the algorithm) according to one criterion can also contribute in the consideration of the other two criteria. There are two cases in which placing a cell (that is, setting the next considered cell to be “occupied”) contributes to two of the criteria. First, polyominoes can be made connected to the top or bottom of the box through cells that are laid along a diagonal of the rotated lattice, and thus, will result later in making the polyominoes wider than their heights. In this case, we need to add only $\max(n_h, n_w)$ cells in order to make polyominoes satisfy both criteria (b) and (c). Second, in the same manner, polyominoes can be made of a single connected component through cells that can also contribute to making them wider than their height. Suppose that in order to turn polyominoes with a specific signature into connected ones in the most “economic” way (using the least amount of cells), we need to add k sequences of cells of respective lengths $(s_1, \dots, s_k)$. (The number k and the individual values s_i depend, of course, on the specific signature.) Then, each such segment can be built as an L-shape on top of the boundary (see Figure 6), and, thus, contribute $\lfloor s_i/2 \rfloor + 1$ to the width of the polyominoes. Let $s^* = \max_{1 \leq i \leq k} s_i$. Putting together the two observations above, we conclude that less than $\max\{n_w, n_w + n_c - (\lfloor s^*/2 \rfloor + 1), n_h + n_c\}$ additional cells will not suffice for turning the partial polyominoes into valid ones, and, hence, entries of signatures that do not have this “budget” (possibly only portions of entries) can be eliminated. Obviously, all these considerations are not applicable to the original version of the algorithm.

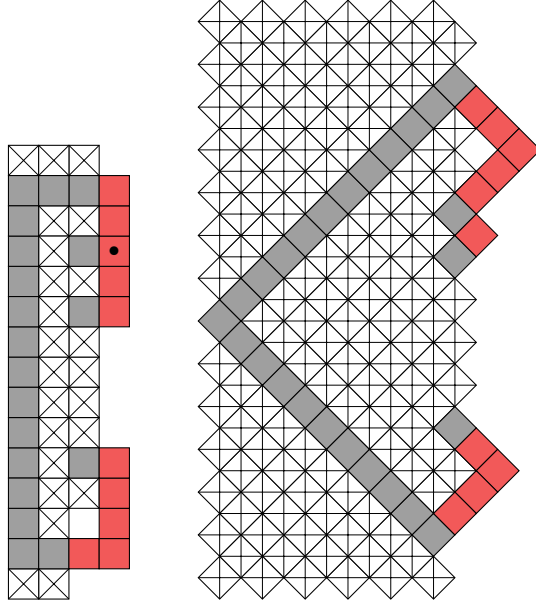


Fig. 6 A partial polyomino in both lattices, with the minimum number of cells (colored in red) needed to make it connected.

Remark. While computing n_h and n_w is straightforward, computing n_c can be somewhat more complicated. In both Jensen's version and our version of the algorithm, we need to find a set of sequences of cells that will turn the current signature into some signature that represents connected polyominoes, while using the minimum possible number of cells. In the original version of the algorithm, two sequences of cells, which will connect two components of a partial polyomino, may overlap in one cell (see, *e.g.*, the cell marked with a bullet ('•') in Figure 6). Computing n_c in this scenario can be solved using a dynamic-programming procedure whose running time is linear with the size of the signature. We assume that a similar approach was used by Jensen [9], although this is not mentioned there explicitly.

This dynamic-programming approach is applicable also in our version of the algorithm, however, in our case, two sequences of cells cannot overlap. This allows another approach, based on modeling the problem as a graph G , in which the nodes are the connected components and the edges are the sequences connecting the components, where the weight of an edge is the length of the sequence. The problem of finding the minimum number of cells is now reduced into finding the minimum spanning tree (MST) of G . Although using an MST algorithm adds to the running time a logarithmic factor (with respect to the size of the signature), it turns out that Kruskal's classical algorithm is in practice slightly faster than the dynamic-programming approach for the small sizes we are dealing with.

4.2 Symmetries

Another method for reducing the number of signatures whose data are stored in the database is utilizing symmetries that show up in the course of the algorithm.

The first type of symmetry arises when we consider different boxes. When the height is even, we have pairs of boxes which are identical up to mirroring about the horizontal axis, *e.g.*, the two lower boxes in Figure 3. This allows us to process only one of these boxes, and then double the final count.

The second type of symmetry arises with individual boxes of odd height. Consider “kinkless” signatures, that is, signatures created when we finish processing a column. If two such distinct signatures are the inverse of each other, then the respective partial polyominoes can be completed into valid polyominoes in the same number of ways. This allows us to merge every such pair of signatures into a single one, effectively halving the number of signatures before starting to process the next column. Note that mirroring signatures is not applicable to boxes of an even height since doing this would project the respective partial polyominoes into another, mirrored, box of the same height. For example, consider any signature in the lower left box in Figure 3. The three cells making the signature are in rows 1, 3, and 5 (starting to count top-down from 0). Flipping it around the horizontal axis would map the signature cells into rows 0, 2, and 4. Hence, it cannot be part of this box.

This improvement is applicable also to the original version of Jensen’s algorithm. However, we did not find a mentioning of this in his paper [9].

4.3 Parallelization

The parallelization scheme that we use is a generalization of the one proposed for the original version of the algorithm. Jensen suggested [9] that at the beginning of processing each column, the signatures be split into sets according to the occupancy pattern of the lower half of the signature, where the “occupancy pattern” is the binary pattern of boundary cells when considering only whether or not they are occupied, disregarding the differences between the 1–4 codes of these cells, that is, ignoring the connectivity to the left of the boundary. Since placing an occupied cell in the top half of the boundary will not change the occupancy pattern in the lower part (which is a nice observation), and, thus, the new signature will be in the same original set, such sets can be processed independently of each other and in parallel until we reach the lower half of the signature. At this stage, the signatures are redistributed into new sets, this time according to the occupancy pattern of the upper half of the pattern, with the same reasoning as above.

In the modified version of the algorithm, we added a parameter that controlled the granularity of the separation into different sets of signatures. When the parameter is set to k , we split the signatures into k parts (instead of only two parts), and redistribute the signatures before processing each chunk of h/k

cells of the boundary, where h is as above the length of the boundary (height of the box). When signatures are redistributed, they are split into sets according to the occupancy pattern of the complementing part of the chunk, which is of size $(k-1)h/k$. Obviously, setting $k=2$ would result in the original parallelization scheme of Jensen.

Using larger values of k incurs a larger time spent on redistributing the signatures between the new sets. However, this modification is eventually beneficial for two reasons.

1. First, the bottleneck of this algorithm in our version is met in a smaller box height. For example, when $n=56$, the most time-consuming box in our implementation was of height 15, while according to Jensen, the most time-consuming box for his implementation was of height 22–24. (We discuss the reasons for this difference in Section 5.) When the signature length is 15 and $k=2$, the patterns are split according to half of the cells, that is, 7 or 8 cells, thus, into only 128 sets in the lesser case. These sets are far from being equal in size. For example, there are many more signatures with an occupancy pattern of 1111111 than patterns with occupancy pattern 0000000, and, thus, when the signature length is too small, we cannot balance the load between the processors.
2. The other reason for this modification is for conserving memory. Each processor processes a single set of signatures at a time. As the non-active sets are being stored in a compressed way, having smaller sets implies that at any given time, signatures that correspond to a smaller amount of patterns is being held in an uncompressed manner, which reduces the memory consumption of the algorithm.

5 Comparison

Some factors play in favor of our version, compared to the original algorithm, while other factors play the opposite role. The bottom line is that our version wins by far for larger polyomino sizes, and we now proceed to explain why.

- *Number of boxes.* Recall that for computing the number of polyominoes of size n , we count polyominoes in boxes of a range of heights. In the original algorithm, heights are in the range $1, \dots, \lceil n/2 \rceil$. The algorithm stops at height $\lceil n/2 \rceil$ since we count only polyominoes whose width is at least their height. Thus, boxes of height greater than $\lceil n/2 \rceil$ contain polyominoes that were already counted. In our version of the algorithm, we cannot apply this optimization. Although it also counts only polyominoes that are wider than their height (in terms of diagonal width and height), it is a fact that boxes of height greater than $\lceil n/2 \rceil$ can contain polyominoes that are wider than their height. For example, both diagonal width and height of the “stick” polyomino of length n () are n , and, therefore, they are counted only within the box of height n . (It is worth mentioning that our version of the algorithm still counts only polyominoes that are wider than their

- height since this helps considerably the pruning process.) Recall that for each height, we have two types of boxes, and, thus, we have a total of $2n$ boxes to iterate through, compared to $\lceil n/2 \rceil$ in the original algorithm.
- *Signature length.* The parameter that affects the running time of a single step the most is the length of the signature. In the original version of the algorithm, each row has a cell participating in the signature, and, thus, the signature length is equal to the height of the box. In our version of the algorithm, the rows are interleaved, and, thus, only half of the rows are represented in the signature, and, hence, the signature length is only half the height of the box. On the other hand, the maximum height of a box in our version is n , compared to $\lceil n/2 \rceil$ in the original algorithm. In conclusion, the maximum signature length is the same in both versions.
 - *Pruning.* As discussed above, there are three quantities that affect the pruning of signatures in the course of the algorithm: The number of cells needed to make the components associated with a specific signature connected, the distance to the top and bottom of the box, and the number of cells needed to make the polyomino wider than its height. For all three parameters, the range of possible values is larger in our version, hence, the pruning is much more efficient. All three parameters, in both versions of the algorithm, are obviously bounded from above by the height of the box. The difference between the two versions is that the height of the box in our version is at most n , while in the original version, it is at most $\lceil n/2 \rceil$. It may seem that this would inhibit our version. However, combined with the fact that the signature length is halved, it turns out that it is in favor of our version since the pruning range is larger and, hence, the pruning is more effective.

Putting together all of the above, we see that there are factors in favor and against each version of the algorithm when performance is compared. Nevertheless, when we implemented both versions with all improvements mentioned above, our version was significantly more efficient for counting polyominoes of larger areas. This was mainly because it processed much less signatures, which are the main factor in the running time of both versions. That is, it turned out that signature pruning in our version was much more “aggressive.” What was the reason for this?

Signature pruning is much more effective on the 45° -rotated lattice than on the regular lattice because more cells (about twice) are needed on the former lattice than on the latter one for closing vertical gaps (see Figure 6): For closing a gap of k cells, $2k + 1$ cells are needed on the rotated lattice, while only $k + 2$ cells are needed on the regular lattice.

Hence, signatures that contain cells with a big vertical gap, who nevertheless belong to the same component, are created in our version of the algorithm much later than in the original version. More importantly, signatures that contain cells with a big vertical gap, and that belong to different components, are pruned out in our version of the algorithm much earlier than in the origi-

Table 2 The total number of signatures processed (after pruning) while computing $A(n)$. In our implementation, our version of the algorithm processes less signatures than Jensen’s original version for $n \geq 23$.

Jensen’s version wins				Our version wins			
n	Jensen’s	Ours	Ratio	n	Jensen’s	Ours	Ratio
3	14	19	1.357	23	2,197,843	2,174,613	0.989
4	21	65	3.095	24	3,427,165	3,225,791	0.941
5	115	160	1.391	25	5,402,306	4,793,829	0.887
6	160	396	2.475	26	8,400,423	7,072,343	0.842
7	525	858	1.634	27	13,114,620	10,392,875	0.792
8	719	1,702	2.367	28	20,303,600	15,271,610	0.752
9	1,956	3,181	1.626	29	31,472,506	22,383,133	0.711
10	2,798	5,819	2.080	30	48,562,989	32,599,124	0.671
11	6,080	10,060	1.655	31	74,873,463	47,398,923	0.633
12	9,305	16,944	1.821	32	114,986,657	68,927,927	0.599
13	17,753	28,114	1.584	33	176,561,372	99,988,249	0.566
14	28,351	45,593	1.608	34	270,108,311	144,385,132	0.535
15	48,871	72,357	1.481	35	413,271,060	208,511,368	0.505
16	79,345	113,324	1.428	36	629,893,445	301,121,651	0.478
17	131,663	177,658	1.349	37	960,526,646	434,209,504	0.452
18	211,907	274,791	1.297	38	1,459,702,899	624,050,985	0.428
19	344,120	418,826	1.217	39	2,218,864,323	895,884,471	0.404
20	545,895	636,957	1.167	40	3,362,894,943	1,282,445,284	0.381
21	878,922	966,832	1.100				
22	1,380,912	1,451,606	1.051				

nal version. This difference between the two versions is demonstrated well in Section 5.

Table 2 and Figure 7 compare the total number of signatures processed by the two versions of the algorithm.² The symmetries improvements, discussed in Section 4.2, were implemented in both versions of the algorithm. It can be seen that for small values of n , Jensen’s algorithm processes less signatures than our version, however, starting at $n = 23$, the ratio between the respective numbers becomes consistently more and more in favor of our version. Note

² The term “total number of signatures” deserves a careful explanation. Upfront, pruning signatures does not reduce their number since a pruned signature needs anyway to be analyzed before being pruned. However, by this term we mean something else. We count a signature θ if the data in the database entry, which contains the counts of all partial polyominoes with boundaries whose signature is θ , are accessed. If a signature is pruned before the data in its entry is accessed, then it is not counted. In addition, what we count is the *total number* of accesses to entries of signatures. That is, if the same signature is processed k times (in different stages of the algorithm), then we add k to the total count.

Another issue that is worth mentioning is that Jensen [8, P. 873, Fig. 2] reports “maximal numbers” of configurations which seem to be smaller than the signature counts that we report. A simple explanation is that Jensen reports the maximum, taken over all individual considered cells (under the kink), during the entire course of the algorithm, while, as mentioned above, we sum up all these numbers.

On top of all this, it is certainly possible that Jensen implemented pruning rules that are more efficient than those we implemented. (However, as much as we can testify, we implemented all rules described in Refs. [8,9] and added more of our own.) Anyway, in order to make a fair comparison between the two versions of the algorithm, we implemented the same pruning rules in both versions, and applied the same signature counting method in both implementations.

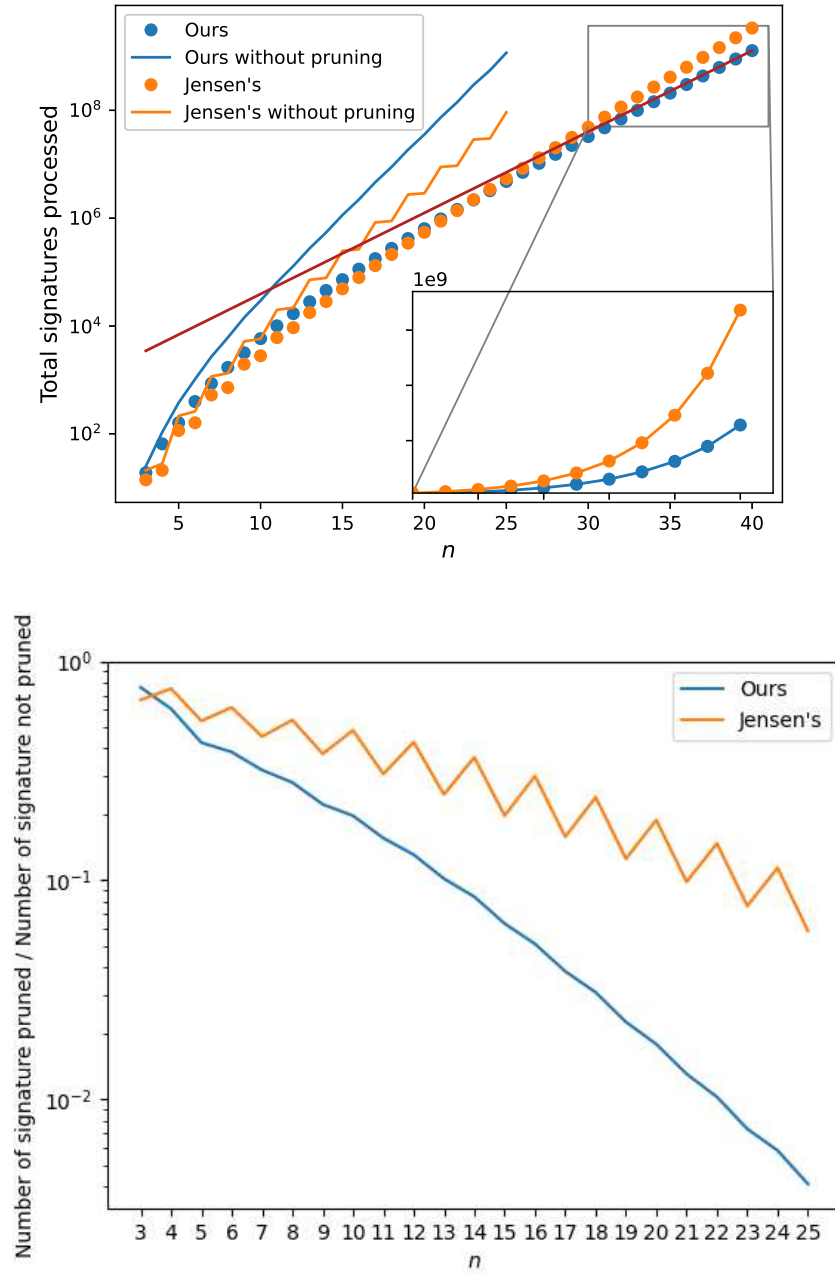


Fig. 7 (Top) The total number of signatures processed (with and without pruning) while computing $A(n)$ (in a logarithmic scale). The red line represents a growth rate of $\sqrt{2}$. The growth rate of our version of the algorithm drops below $\sqrt{2}$ at $n = 49$. The zoomed-in area is displayed in a regular scale. (Bottom) Ratios of signature pruning in the two versions.

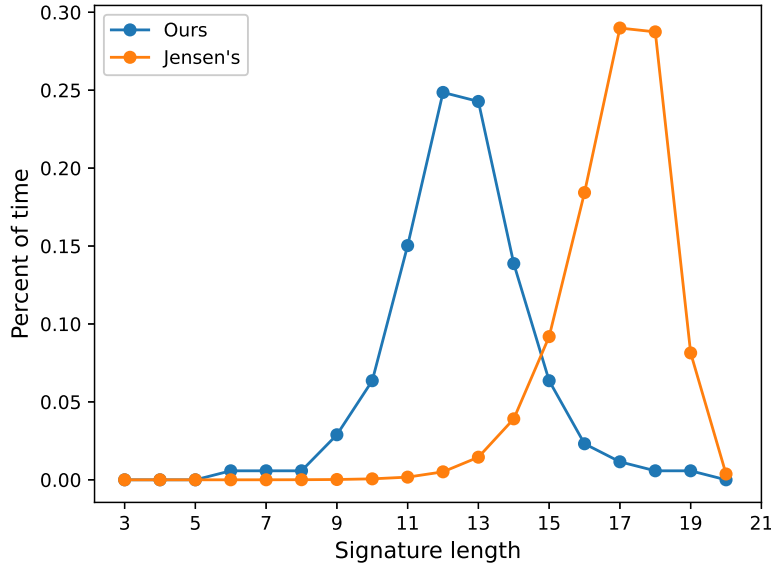


Fig. 8 Percent of time spent for each signature length when computing $A(40)$.

that in the original version, the curve plotting the numbers of signatures is “jumpy” since a new box appears only in odd heights, so that in even sizes, the original number of signatures does not change much but less signatures are pruned with respect to the previous size.

It is also worth mentioning that Jensen claimed that the number of signatures *after pruning*, when computing $A(n)$, behaved proportionally to $\sqrt{2}^n$. (It was shown later [2] that the number of signatures *without pruning* was proportional to $\sqrt{3}^n$.) In our implementation, the growth rate of the number of signatures drops below $\sqrt{2}$ at $n = 49$, and it then continues to drop for larger values of n . This leads us to suspect that it is only a coincidence that the behavior of this sequence is roughly like $\sqrt{2}^n$ for a considerable amount of the leading terms.

Another measure of comparison is the signature length in which the running time reaches a peak (for a given value of n), that is, what box takes the longest time to compute. Jensen reported that for computing $A(56)$, the boxes that required the longest running time were of heights in the range 22–24. In our version, when we computed $A(56)$, the running time reached a peak for boxes of height in the range 30–32, which translates (as already explained) into signatures of size 15–16. The reason for the peak being earlier (at smaller heights) is that for larger boxes, the pruning is much more efficient, as discussed above. Figure 8 shows this behavior for the computation of $A(40)$. While this is not a direct measure of the efficiency of the algorithm, it provides an

explanation for why our version processes less signatures and is able to count polyominoes of larger sizes. As explained above, it prunes signatures in the larger boxes more efficiently, and, thus, most of the running time is spent on the smaller boxes, which naturally correspond to much less signatures.

6 Results

We implemented the improved version of the algorithm in the C++ programming language. The computation of $A(70)$ took about 15,000 CPU hours (21 months) on a 32-CPU machine with 32GB of RAM. Table 3 provides the counts of polyominoes computed by our program. For sizes $1 \leq n \leq 56$, the values computed for $A(n)$ agree with previously published values [9], while the values for $57 \leq n \leq 70$ are new.

Let us attempt to estimate for which value of n , our version of the algorithm processes the same amount of signatures as does Jensen's version for computing $A(56)$ (the previously last known element). Knowing that for $n = 40$, the ratio of signatures processed by the two versions of the algorithm is around 0.381 (see Table 2), and that the growth rates of numbers of signatures are almost steady (but slightly monotone decreasing) around 1.5 and $\sqrt{2}$, respectively, we solve for s the equation $x \cdot 1.5^{16} = (0.381 \cdot x)(\sqrt{2})^s$, and find that $s \approx 21.5$. Since the gap between the two sequences has a slow tendency of growing, we round s up to 22, or even 23, and estimate that our version of the algorithm processes roughly the same amount of signatures at $n = 40 + 23 = 63$ as Jensen's version does for $n = 56$. Hence, we attribute, out of the 14 new terms of $A(n)$, about seven to our improved algorithm, and the other seven to other factors, such as having better hardware, different implementations, and available running time.

Remarks

1. In 1999, Madras [12] showed the existence of the growth rate of polyominoes, the limit $\lambda := \lim_{n \rightarrow \infty} (A(n+1)/A(n))$, and the currently best-known lower bound on λ is 4.00253 [3]. It is known *empirically*, but has never been shown analytically, that the sequence $(A(n+1)/A(n))$ is monotone increasing. Had this been proven rigorously, we would have obtained $A(70)/A(69) > 4.00512$ as an improved lower bound on λ .
2. In a private communication, Iwan Jensen applied two numerical methods and used the 14 new values of $A(n)$ for obtaining 4.06256912(2) and 4.06256903(5) as improved estimates of λ .
3. Günter Rote drew our attention to the fact that in a lecture in 2014, D.E. Knuth [11] had used diagonals on the square lattice for deriving generating functions for the counts of polyominoes in given bounding boxes, and provided enumeration of polyominoes of area 47 (rounded to units of 10^{22}) by their bounding boxes. About 14 years earlier, Knuth presented

Table 3 Counts of polyominoes.

n (size)	$A(n)$	n (size)	$A(n)$
1	1	21	88,983,512,783
2	2	22	345,532,572,678
3	6	23	1,344,372,335,524
4	19	24	5,239,988,770,268
5	63	25	20,457,802,016,011
6	216	26	79,992,676,367,108
7	760	27	313,224,032,098,244
8	2,725	28	1,228,088,671,826,973
9	9,910	29	4,820,975,409,710,116
10	36,446	30	18,946,775,782,611,174
11	135,268	31	74,541,651,404,935,148
12	505,861	32	293,560,133,910,477,776
13	1,903,890	33	1,157,186,142,148,293,638
14	7,204,874	34	4,565,553,929,115,769,162
15	27,394,666	35	18,027,932,215,016,128,134
16	104,592,937	36	71,242,712,815,411,950,635
17	400,795,844	37	281,746,550,485,032,531,911
18	1,540,820,542	38	1,115,021,869,572,604,692,100
19	5,940,738,676	39	4,415,695,134,978,868,448,596
20	22,964,779,660	40	17,498,111,172,838,312,982,542
41			69,381,900,728,932,743,048,483
42			275,265,412,856,343,074,274,146
43			1,092,687,308,874,612,006,972,082
44			4,339,784,013,643,393,384,603,906
45			17,244,800,728,846,724,289,191,074
46			68,557,762,666,345,165,410,168,738
47			272,680,844,424,943,840,614,538,634
48			1,085,035,285,182,087,705,685,323,738
49			4,319,331,509,344,565,487,555,270,660
50			17,201,460,881,287,871,798,942,420,736
51			68,530,413,174,845,561,618,160,604,928
52			273,126,660,016,519,143,293,320,026,256
53			1,088,933,685,559,350,300,820,095,990,030
54			4,342,997,469,623,933,155,942,753,899,000
55			17,326,987,021,737,904,384,935,434,351,490
56			69,150,714,562,532,896,936,574,425,480,218
New Values			
57			276,061,302,869,769,053,815,091,348,274,853
58			1,102,414,654,388,614,817,828,362,087,885,194
59			4,403,627,610,727,810,528,935,609,181,494,038
60			17,595,360,125,786,076,902,429,902,468,975,094
61			70,324,022,977,839,757,717,649,768,794,923,974
62			281,140,509,936,541,236,506,957,176,030,736,610
63			1,124,226,206,002,101,486,403,009,097,923,067,462
64			4,496,670,726,609,716,846,990,603,851,802,851,046
65			17,990,046,482,672,581,050,565,325,516,788,100,067
66			71,990,329,116,147,598,315,672,557,624,308,271,186
67			288,146,737,936,083,547,752,609,866,746,184,117,546
68			1,153,580,914,959,613,979,832,549,218,358,894,513,290
69			4,619,282,047,583,828,929,546,825,973,053,580,643,926
70			18,500,792,645,885,711,270,652,890,811,942,343,400,814

in a lecture³ a similar method for building polyominoes in diagonals, where the idea was adding an entire diagonal layer instead of cell by cell.

7 Conclusion

In this paper, we suggest a substantial improvement to Jensen's algorithm for counting polyominoes. Using a program that implemented this improvement, we were able to extend the count of polyominoes from 56 terms to the first 70 terms of the sequence $(A(n))$.

Future research directions include introducing more programming tricks which will allow us to compute more values of $A(n)$ and generalizing the method to three dimensions.

Acknowledgements The authors would like to thank Günter Rote for helpful comments on a preliminary version of this paper, and Iwan Jensen for helpful discussions about the contents of this paper.

Conflict of Interest

The authors declare that they have no conflict of interest.

References

1. Barequet, G., Ben-Shachar, G.: Counting polyominoes, revisited. In: 2024 Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX), pp. 133–143. SIAM (2024)
2. Barequet, G., Moffie, M.: On the complexity of Jensen's algorithm for counting fixed polyominoes. *Journal of Discrete Algorithms* **5**(2), 348–355 (2007)
3. Barequet, G., Rote, G., Shalah, M.: $\lambda > 4$: An improved lower bound on the growth constant of polyominoes. *Comm. of the ACM* **59**(7), 88–95 (2016)
4. Barequet, G., Shalah, M.: Improved upper bounds on the growth constants of polyominoes and polycubes. In: *Latin American Symposium on Theoretical Informatics*, pp. 532–545. Springer (2021). A full version to appear in *Algorithmica*
5. Broadbent, S., Hammersley, J.: Percolation processes: I. Crystals and Mazes. In: *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 53, pp. 629–641. Cambridge University Press (1957)
6. Conway, A.: Enumerating 2d percolation series by the finite-lattice method: Theory. *Journal of Physics A: Mathematical and General* **28**(2), 335–349 (1995)
7. Jacobsen, J., Guttmann, A.: *Polygons, Polyominoes and Polycubes*. Springer Dordrecht (2009)
8. Jensen, I.: Enumerations of lattice animals and trees. *Journal of statistical physics* **102**, 865–881 (2001)
9. Jensen, I.: Counting polyominoes: A parallel implementation for cluster computing. In: P. Sloot, D. Abramson, A. Bogdanov, Y. Gorbachev, J. Dongarra, A. Zomaya (eds.) *Computational Science — ICCS 2003*, pp. 203–212. Springer Berlin Heidelberg, Berlin, Heidelberg (2003)
10. Klarner, D.: Cell growth problems. *Canadian Journal of Mathematics* **19**, 851–863 (1967)

³ See the YouTube video https://www.youtube.com/watch?v=df_7hydjaT0.

11. Knuth, D.: Problems that Philippe would have loved. Presented in AofA'14 in memory of F. Flajolet, available at <https://www-cs-faculty.stanford.edu/~knuth/papers/flaj2014.pdf> (2014)
12. Madras, N.: A pattern theorem for lattice clusters. *Annals of Combinatorics* **3**(2), 357–384 (1999)
13. Redelmeier, D.: Counting polyominoes: yet another attack. *Discrete Mathematics* **36**(2), 191–203 (1981)
14. Sloane, N., et al.: The on-line encyclopedia of integer sequences. Published electronically at <https://oeis.org> (2018)